



**TECNOLÓGICO
NACIONAL DE MÉXICO®**

**INSTITUTO TECNOLÓGICO DE CULIACAN
PI-U3-4-FLORESURQUIDY**

Alumno: Sergio Eliel Flores Urquidy

No. Control: 25171184

Ing . Sistemas computacionales

Maestro: MARIA LUCIA BARRON ESTRADA

Materia: PROGRAMACION OREINTADA A OBJETOS

Culiacán Sinaloa

Fecha: 04/05/2026

ÍNDICE

1. INTRODUCCION
2. Tarea 1 – -----¿Qué puedes hacer en una subclase?
3. Tarea 6 – -----Error por paquetes distintos
4. Tarea 7 – -----Constructores en clases padre e hijo
5. Tarea 2 (U4) – -----Error en el método equals
6. CONCLUSION

INTRODUCCIÓN

Cuando empecé a ver estas unidades la verdad no tenía muy claro para qué servía la herencia, pero con los ejemplos y las tareas de programación se fue entendiendo mucho mejor. La herencia básicamente te permite crear clases nuevas a partir de otras ya existentes, reutilizando sus atributos y métodos sin tener que escribir todo desde cero. Ya en la unidad 4 vimos el polimorfismo, que va muy de la mano con lo anterior, y que permite que los objetos se comporten de formas distintas según el contexto, usando herramientas como la redefinición de métodos, las clases abstractas y las interfaces. En general estas dos unidades me ayudaron a entender por qué la programación orientada a objetos es tan usada, ya que hace el código más organizado, flexible y fácil de mantener.

TAREA 1

1. ¿Qué puedes hacer en una subclase? Puede usar los atributos públicos y protegidos que heredó directamente, también puede declarar nuevos atributos y métodos, se pueden sobrescribir los métodos (redefinir), y se puede invocar al constructor de la superclase en la subclase.
2. ¿Cómo puedes acceder desde una subclase a los miembros privados de la superclase? Mediante los métodos getter y setter o por las clases anidadas.
3. Explica un ejemplo de Cast: Ejemplo de la bicicleta de montaña que se encuentra en Oracle. Se tiene una bicicleta de montaña pero está guardada o tomada pensando que es una bicicleta normal, con el cast lo que se le dice a Java es que trate a esta bicicleta como si fuera una bicicleta de montaña. En tiempo de ejecución, Java verifica que el objeto sea realmente de ese tipo y si no lo es lanza error.

TAREA 6

El error que se observa es que las clases están en distinto paquete. Para solucionarlo se debería importar el paquete de la clase de esta forma: `import algunPaquete.Padre`. Además, el primer método funciona porque `p` es tipo hijo y está en la subclase, el segundo método no funciona porque es `protected` y está en otro paquete y no lo permite.

TAREA 7

Tarea 7-A

¿Qué pasa cuando la clase padre no tiene constructor explícito y la clase hijo tampoco? Java le da por defecto un constructor vacío al hijo y llama implícitamente a `super()` (constructor de superclase).

¿Qué pasa cuando el hijo tiene un constructor con parámetros? Java mete un `super()` de manera implícita de igual forma.

Tarea 7-B

¿Qué pasa cuando la clase padre tiene constructor y la clase hijo tiene o no un constructor explícito? Si la superclase tiene constructor sin parámetros no pasa nada, Java llama `super()` implícitamente. Si la superclase tiene parámetros y el hijo no tiene constructor, manda un error de compilación. Si la superclase tiene parámetros y el hijo también, se tiene que escribir el `super` explícito o si no falla.

TAREA 2 – U4

El error se debe a que al usar el equals se comparan 2 objetos: el objeto que recibe como parámetro lo castea para que sea un punto y poder compararlo, pero debido a que p3 es una Persona y no un punto, el programa lanza error al querer castear. Se puede prevenir con un instanceof para verificar que el objeto sea un punto antes de ser casteado.

```
public boolean equals(Object o) {  
    if (!(o instanceof Point)) {  
        return false;  
    }  
    Point p = (Point) o;  
    return ((x == p.x) && (y == p.y));  
}
```

CONCLUSIONES

Después de trabajar en estas dos unidades puedo decir que mi forma de programar cambió bastante. Lo que más me costó de la herencia fue entender el comportamiento de los constructores en clases derivadas, porque al principio me confundía con cuándo Java llama al constructor del padre y cuándo no. Con las tareas de práctica eso fue quedando más claro. Del polimorfismo, lo que más me gustó fue trabajar con clases abstractas e interfaces, ya que permiten definir estructuras generales y darle mucha flexibilidad al programa. En conclusión, la herencia y el polimorfismo son conceptos que al principio parecen complicados, pero una vez que los entiendes sientes que programar se vuelve mucho más poderoso y con más sentido.